

Time Complexity
Best | Average | Worst
= $O(n)$

CONVERTING INFIX EXPRESSION TO POSTFIX EXPRESSION

- converts an infix expression (operators in between operands) into postfix expression (operands after operands) using Stack and precedence rules.

Space Complexity
→ $O(n)$
→ Auxiliary Space (Stack)

1. Infix Expression

- Standard way of writing expressions.
- Operators are between operands.
- Example:
 $A + B * (C - D) / E$.

2. Stack Initialization

- Create an empty stack.
- Create an empty output string
- Stack is used to store operators temporarily.

3. Scan the Expression

- Read infix from left to right
- Process one symbol at a time
- Symbols are of 3 types:
 - Operand
 - Operator
 - Parenthesis

4. Processing Rules

- Operand → Add to output.
- '(' → Push to stack
- '/' → Pop to output until '('.
- Operator → Pop from stack to output while top has higher or equal precedence. Then push current operator.

5. Precedence of Operators

Operator	Precedence
()	-
^	3
* /	2
+ -	1

• Higher precedence evaluated first

b. Empty the stack

- After scanning entire expression, pop all remaining operators from stack and add to output.
- Stack should be empty at the end.

7. Postfix Expression

- The output string obtained is the postfix expression.
- Operators appear after their operands
- Example:
 $ABCD - * + E /$

Example Conversion

Infix → $A + B * (C - D) / E$
Postfix → $ABCD - * + E /$

Steps (Brief):

1. A → Output
2. B → Output
3. + → Stack

4. * → Stack
5. (→ Stack
6. C → Output
7. - → Stack

8. D → Output
9.) → Pop until '('
10. / → Stack
11. E → Output
12. Pop remaining operators

Final Output:
 $ABCD - * + E /$

Efficiency

• Easy conversion for evaluation

• Useful in compilers

• Eliminates need for parentheses

• Postfix evaluation is easy (using stack)

• Time efficient with linear complexity

RECURRENCE RELATION ANALYSIS

Time Complexity:

- Derived from recurrence
- Best | Average | Worst = $\Theta(n \log n)$

Used to solve recursive algorithms systematically without directly applying the Master Theorem.

Space Complexity:

- Depends on recursion depth
- Stack Space = $O(\log n)$ (for balanced recursion)

1. Recurrence Relation

- Mathematical model of recursion.
- Example $T(n) = 2T(n/2) + n$
- Recursive calls + extra work

2. Expansion

- Expand level by level
- Observe repeated terms
- Continue until base case
- Example: $T(n) = 2T(n/2) + n$
 $= 4T(n/4) + 2n$
 $= 8T(n/8) + 3n$
 $\dots$

3. Pattern Recognition

- Identify repeated structure
- Base case: $n/2^k = 1$
 $\Rightarrow k = \log_2 n$
- Pattern: - $T(n) = 2^k T(1) + kn$

4. Summation

- Total work: $\frac{n + n + n + \dots}{(\log n \text{ levels})}$
 $= \sum n$
 $= n \log n$
- Repeated addition
- Use summation formula

5. Final Complexity

- Substitute $k = \log_2 n$
- Therefore, $T(n) = 2^k T(1) + kn$
 $= n \log n$
 $= \Theta(n \log n)$
- Final Time Complexity: $\Theta(n \log n)$

6. Substitution Method

- Assume the solution
- Substitute into recurrence
- Verify the result
- Confirms the pattern.
- Example: Assume $T(n) = O(n \log n)$
- Substitute in $T(n) = 2T(n/2) + n$
- Verify both sides.

Efficiency

- Identifies recurrence patterns
- Simplifies analysis
- Avoids direct Master theorem
- Improves solving efficiency